

21. Sharpening using Laplacian Mask with Diagonal Neighbors

Aim

To sharpen an image using a Laplacian mask including diagonal neighbors.

Program

```
import cv2

import numpy as np

img = cv2.imread("image.jpg")

kernel = np.array([[1, 1, 1],
                   [1, -8, 1],
                   [1, 1, 1]])

laplacian = cv2.filter2D(img, -1, kernel)
sharpened = cv2.subtract(img, laplacian)

cv2.imshow("Original", img)
cv2.imshow("Sharpened", sharpened)

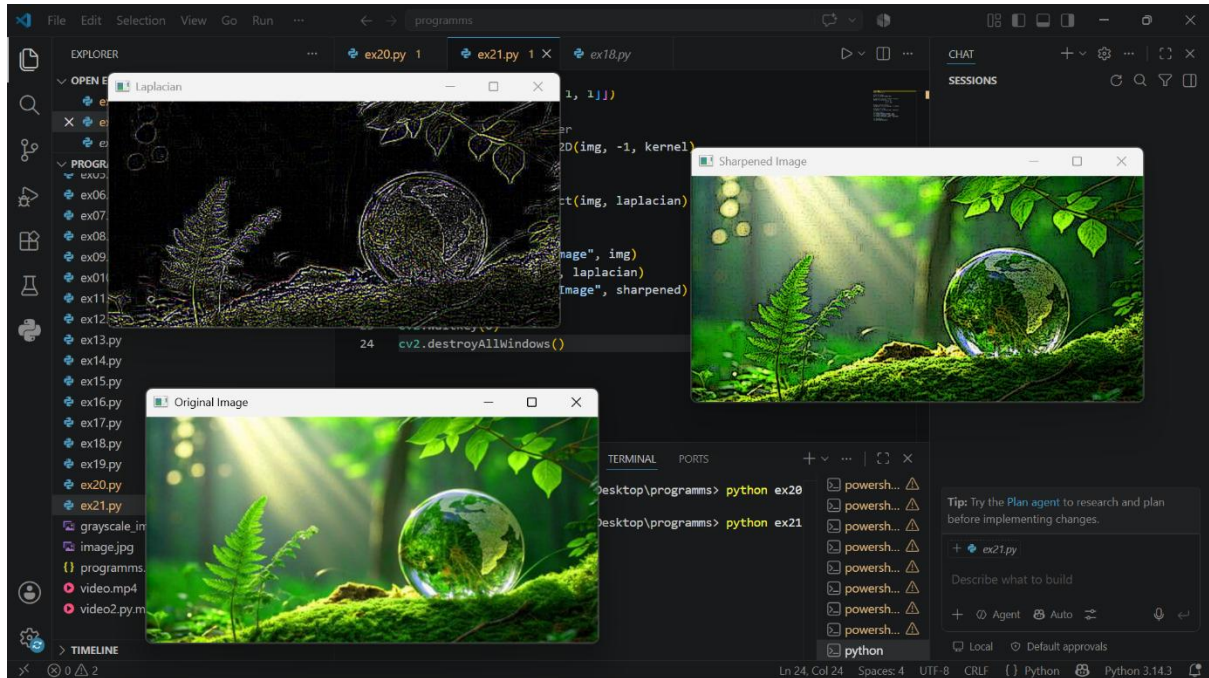
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Define the 3×3 Laplacian mask.
3. Apply the mask using convolution.
4. Subtract the Laplacian from the original.
5. Display the result.

Result

The image was successfully sharpened using the Laplacian mask with diagonal neighbors.



22. Sharpening using Laplacian Mask with Positive Center

Aim

To sharpen an image using a Laplacian mask with a positive center coefficient.

Program

```
import cv2

import numpy as np

img = cv2.imread("image.jpg")

kernel = np.array([[0, -1, 0],
                   [-1, 5, -1],
                   [0, -1, 0]])

sharpened = cv2.filter2D(img, -1, kernel)

cv2.imshow("Original", img)
cv2.imshow("Sharpened", sharpened)

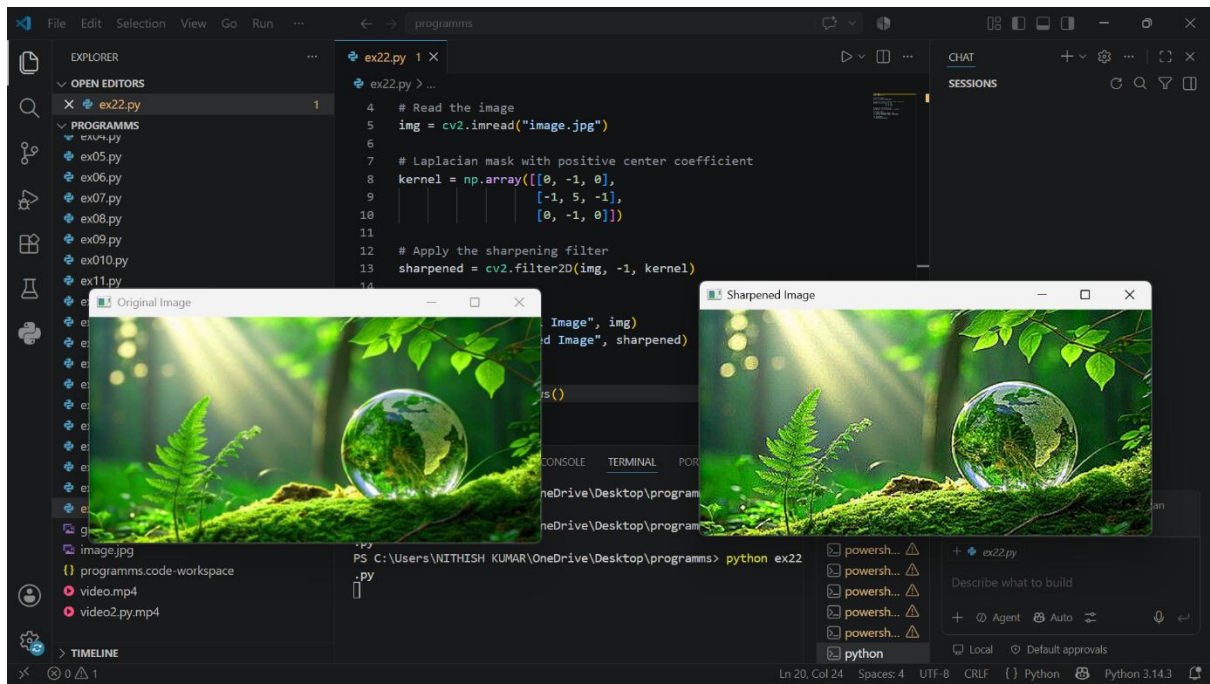
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Define the positive-center mask.
3. Apply the mask using filter2D().
4. Display the sharpened image.

Result

The image was successfully sharpened using the positive-center Laplacian mask.



23. Sharpening using Unsharp Masking

Aim

To sharpen an image using the unsharp masking technique.

Program

```
import cv2

img = cv2.imread("image.jpg")

blur = cv2.GaussianBlur(img, (5, 5), 0)

sharpened = cv2.addWeighted(img, 1.5, blur, -0.5, 0)

cv2.imshow("Original", img)
cv2.imshow("Blurred", blur)
cv2.imshow("Sharpened", sharpened)

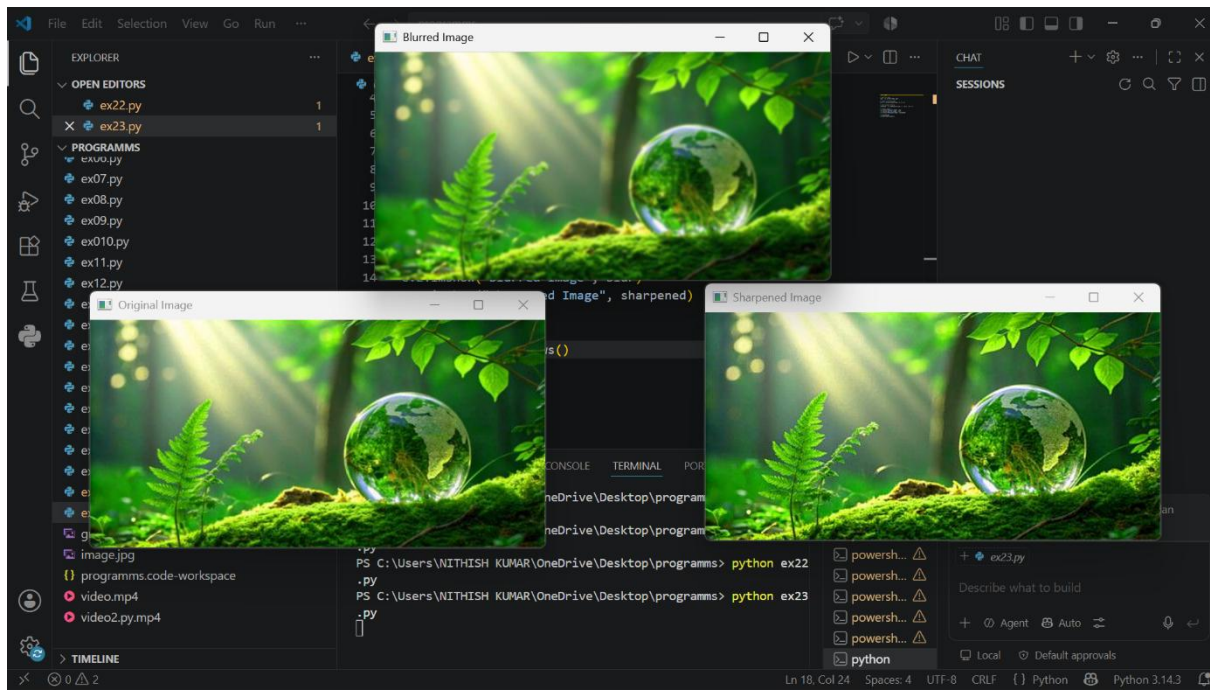
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Create a blurred version using Gaussian Blur.
3. Subtract the blurred image from the original using weighted addition.
4. Display the sharpened image.

Result

The image was successfully sharpened using unsharp masking.



24. Sharpening using High-Boost Mask

Aim

To sharpen an image using a high-boost filtering mask.

Program

```
import cv2

import numpy as np

img = cv2.imread("image.jpg")

A = 2

kernel = np.array([[0, -1, 0],
                   [-1, A + 4, -1],
                   [0, -1, 0]])

sharpened = cv2.filter2D(img, -1, kernel)

cv2.imshow("Original", img)
cv2.imshow("High Boost", sharpened)

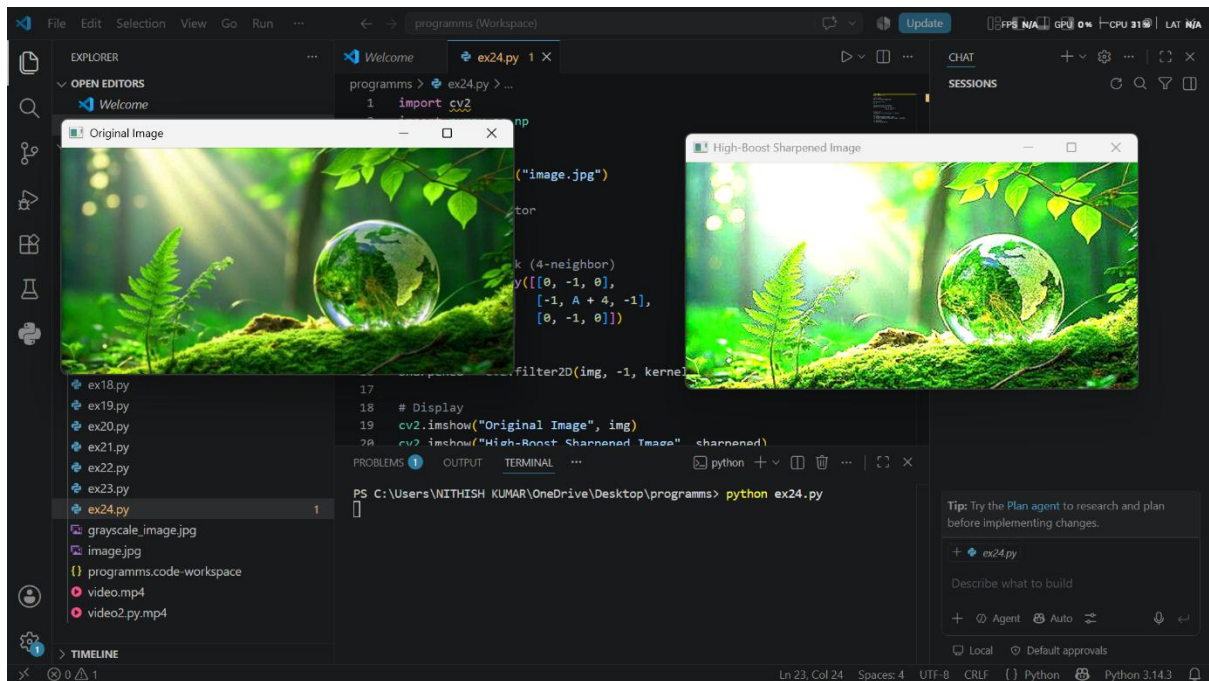
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Select high-boost factor A.
3. Create the high-boost mask.
4. Apply the mask using convolution.
5. Display the sharpened image.

Result

The image was successfully sharpened using high-boost filtering.



25. Sharpening using Gradient Masking

Aim

To sharpen an image using gradient masks.

Program

```
import cv2

import numpy as np

img = cv2.imread("image.jpg", 0)

Gx = np.array([[[-1, 0, 1],
                 [-2, 0, 2],
                 [-1, 0, 1]])

Gy = np.array([[[-1, -2, -1],
                 [0, 0, 0],
                 [1, 2, 1]])

gx = cv2.filter2D(img, cv2.CV_64F, Gx)
gy = cv2.filter2D(img, cv2.CV_64F, Gy)

gradient = cv2.magnitude(
    gx.astype(np.float32),
    gy.astype(np.float32)
)

gradient = cv2.convertScaleAbs(gradient)

sharpened = cv2.add(img, gradient)
```

```
cv2.imshow("Original", img)
```

```
cv2.imshow("Gradient", gradient)
```

```
cv2.imshow("Sharpened", sharpened)
```

```
cv2.waitKey(0)
```

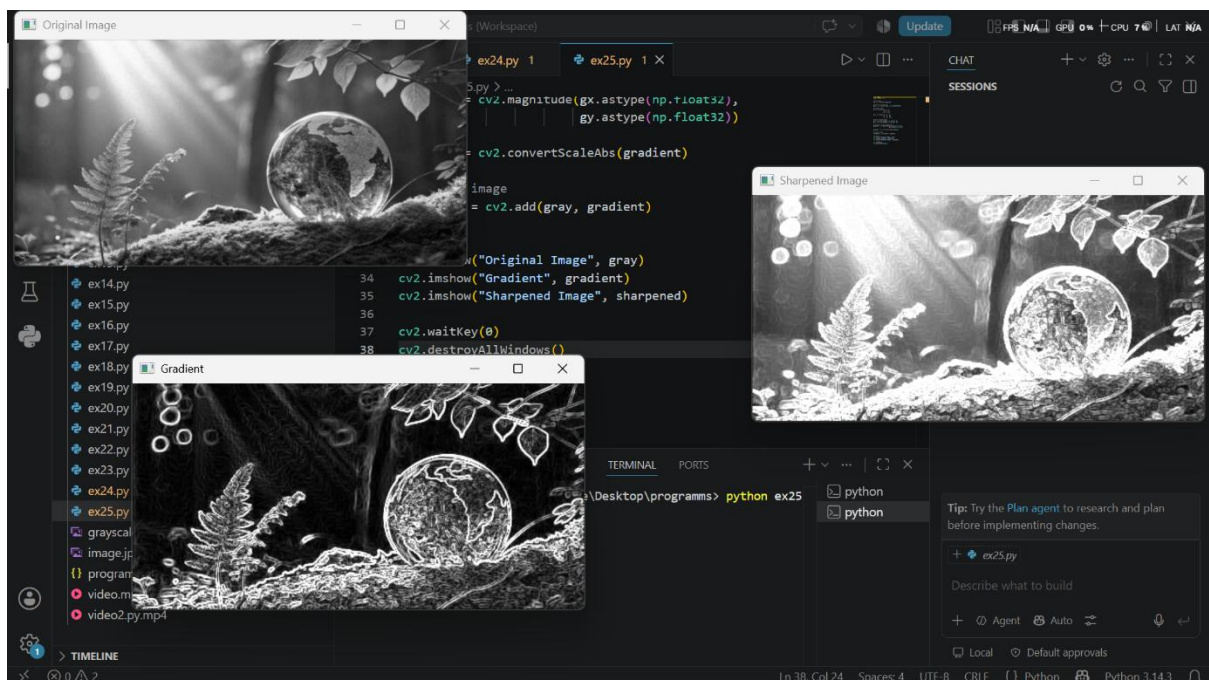
```
cv2.destroyAllWindows()
```

Procedure

1. Read the image in grayscale.
2. Apply X and Y gradient masks.
3. Calculate gradient magnitude.
4. Add the gradient to the original image.
5. Display the sharpened image.

Result

The image was successfully sharpened using gradient masking.



26. Insert Watermark into Image

Aim

To insert a watermark into an image using OpenCV.

Program

```
import cv2

img = cv2.imread("image.jpg")

cv2.putText(img, "WATERMARK",
            (50, 50),
            cv2.FONT_HERSHEY_SIMPLEX,
            1, (255, 255, 255), 2)

cv2.imshow("Watermarked Image", img)

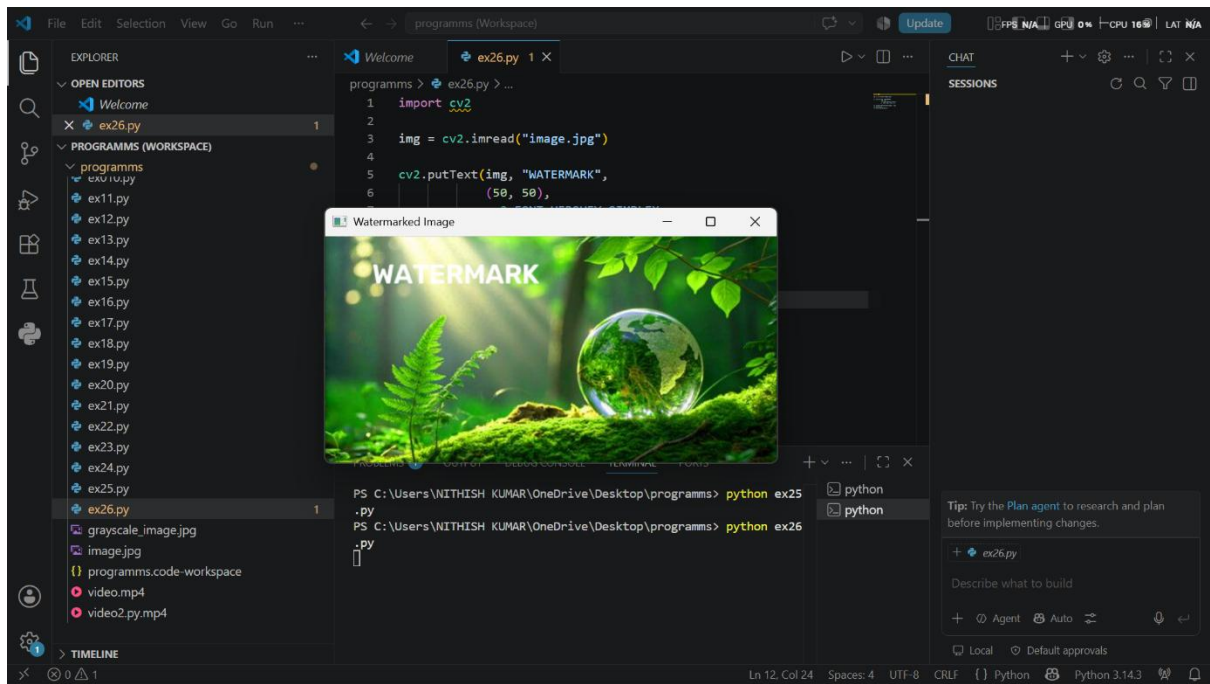
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Add text using `putText()`.
3. Display the watermarked image.

Result

The watermark was successfully inserted into the image.



27. Cropping, Copying and Pasting Image

Aim

To crop an image region and paste it at another location.

Program

```
import cv2

img = cv2.imread("image.jpg")

crop = img[50:200, 50:200]

h, w = crop.shape[:2]

img[250:250+h, 50:50+w] = crop

cv2.imshow("Result", img)
cv2.imshow("Cropped", crop)

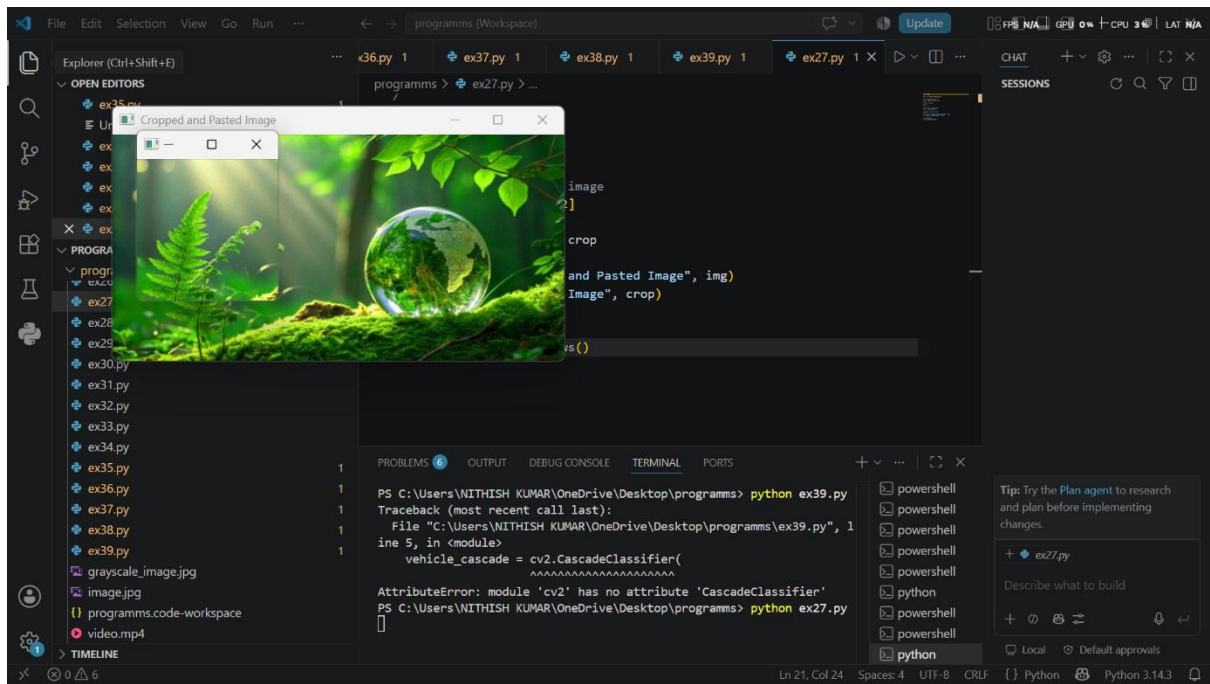
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Select a region using slicing.
3. Copy the selected region.
4. Paste it at another valid location.
5. Display the result.

Result

The image was successfully cropped, copied and pasted.



28. Find Boundary using Convolution Kernel

Aim

To find the boundary of objects using a convolution kernel.

Program

```
import cv2

import numpy as np

img = cv2.imread("image.jpg", 0)

kernel = np.array([[[-1, -1, -1],
                    [-1, 8, -1],
                    [-1, -1, -1]]])

boundary = cv2.filter2D(img, -1, kernel)

cv2.imshow("Original", img)
cv2.imshow("Boundary", boundary)

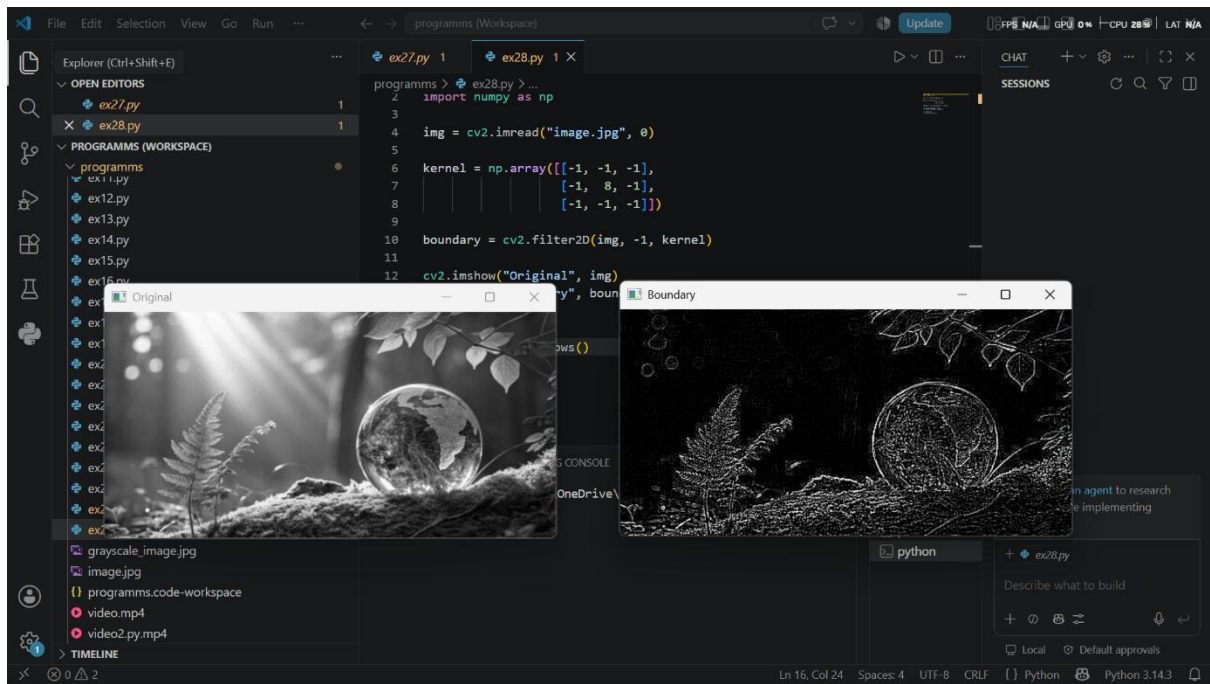
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image in grayscale.
2. Define the boundary detection kernel.
3. Apply convolution.
4. Display the detected boundary.

Result

The boundaries of objects were successfully detected.



29. Morphological Operation – Erosion

Aim

To perform erosion on an image using OpenCV.

Program

```
import cv2

import numpy as np

img = cv2.imread("image.jpg", 0)

kernel = np.ones((5, 5), np.uint8)

erosion = cv2.erode(img, kernel, iterations=1)

cv2.imshow("Original", img)
cv2.imshow("Erosion", erosion)

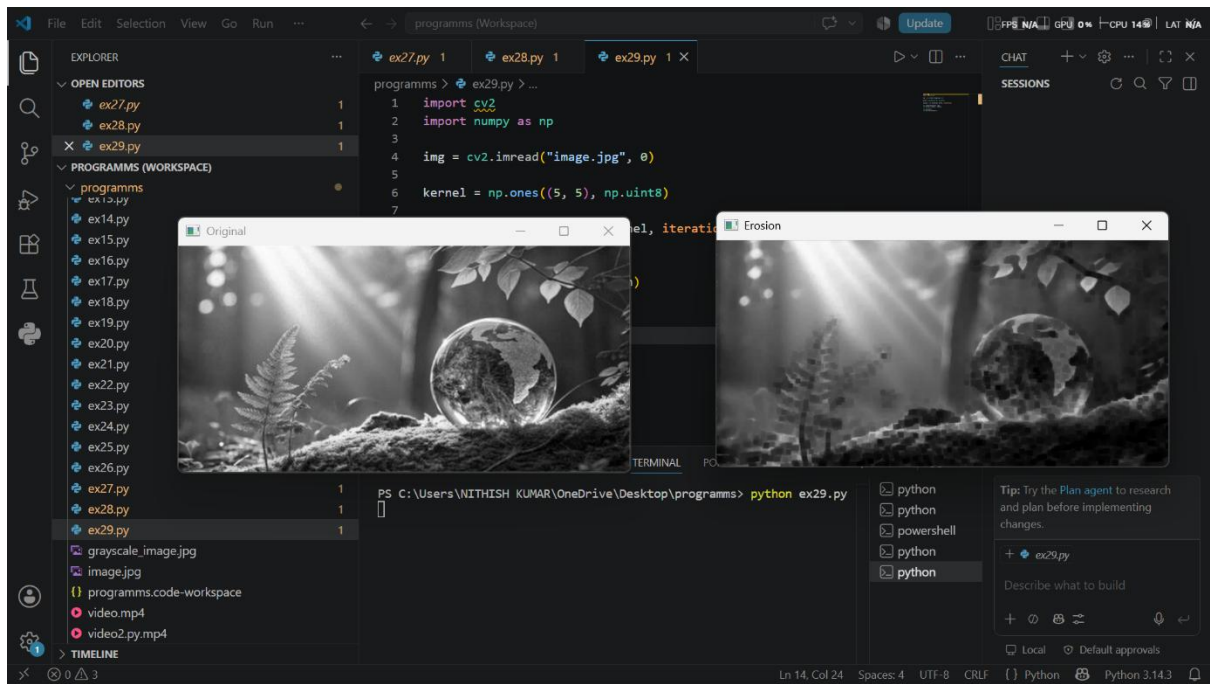
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Create a structuring element.
3. Apply erosion.
4. Display the output.

Result

The objects in the image were successfully eroded.



30. Morphological Operation – Dilation

Aim

To perform dilation on an image using OpenCV.

Program

```
import cv2

import numpy as np

img = cv2.imread("image.jpg", 0)

kernel = np.ones((5, 5), np.uint8)

dilation = cv2.dilate(img, kernel, iterations=1)

cv2.imshow("Original", img)
cv2.imshow("Dilation", dilation)

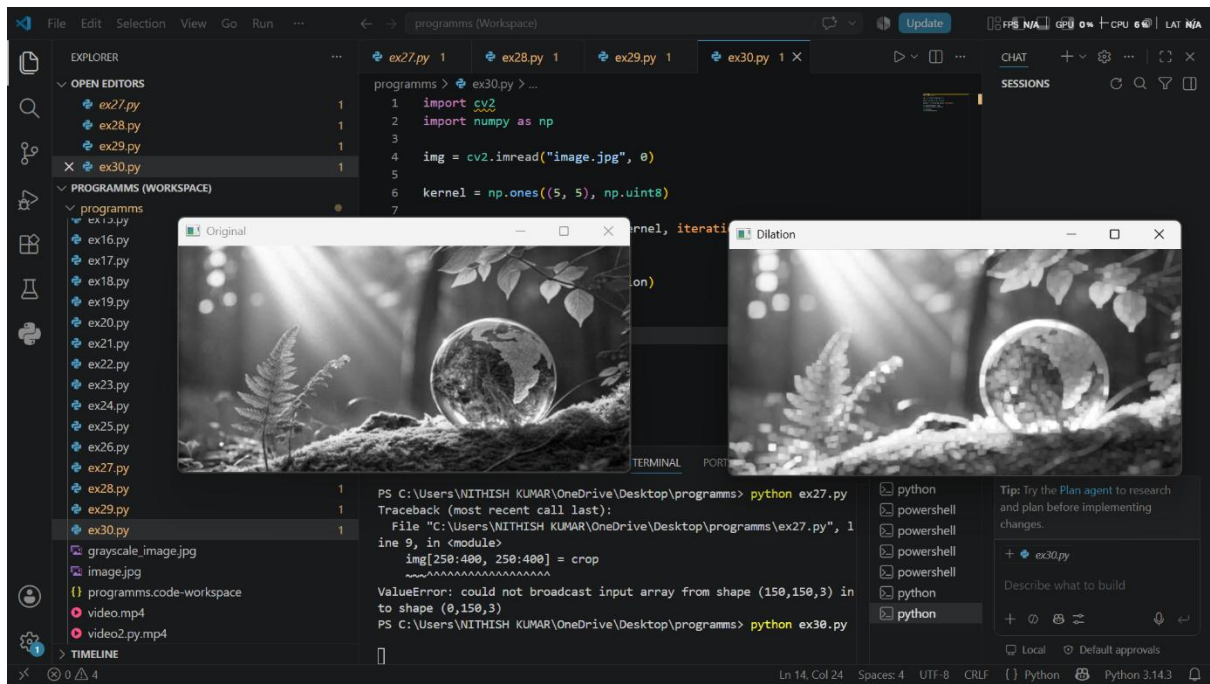
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Create a structuring element.
3. Apply dilation.
4. Display the result.

Result

The objects were successfully dilated.



31. Morphological Operation – Opening

Aim

To perform morphological opening on an image.

Program

```
import cv2

import numpy as np

img = cv2.imread("image.jpg", 0)

kernel = np.ones((5, 5), np.uint8)

opening = cv2.morphologyEx(
    img, cv2.MORPH_OPEN, kernel
)

cv2.imshow("Opening", opening)

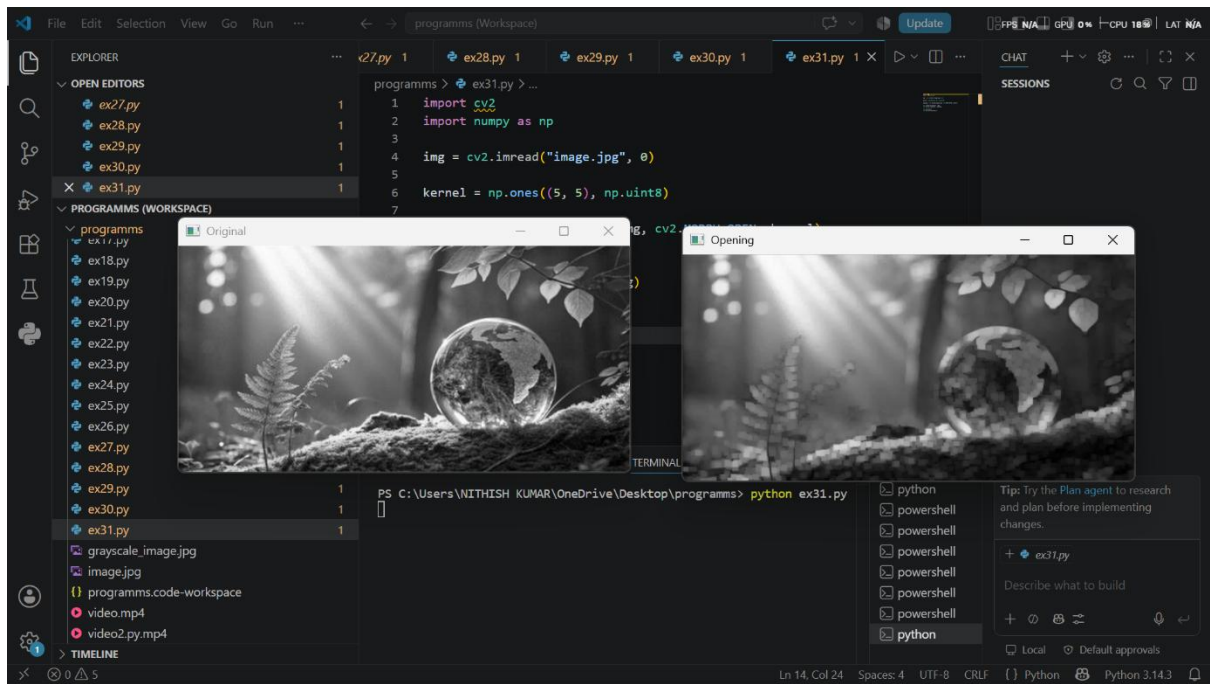
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Create a structuring element.
3. Perform erosion followed by dilation.
4. Display the output.

Result

Small unwanted noise was successfully removed using opening.



32. Morphological Operation – Closing

Aim

To perform morphological closing on an image.

Program

```
import cv2

import numpy as np

img = cv2.imread("image.jpg", 0)

kernel = np.ones((5, 5), np.uint8)

closing = cv2.morphologyEx(
    img, cv2.MORPH_CLOSE, kernel
)

cv2.imshow("Closing", closing)

cv2.waitKey(0)

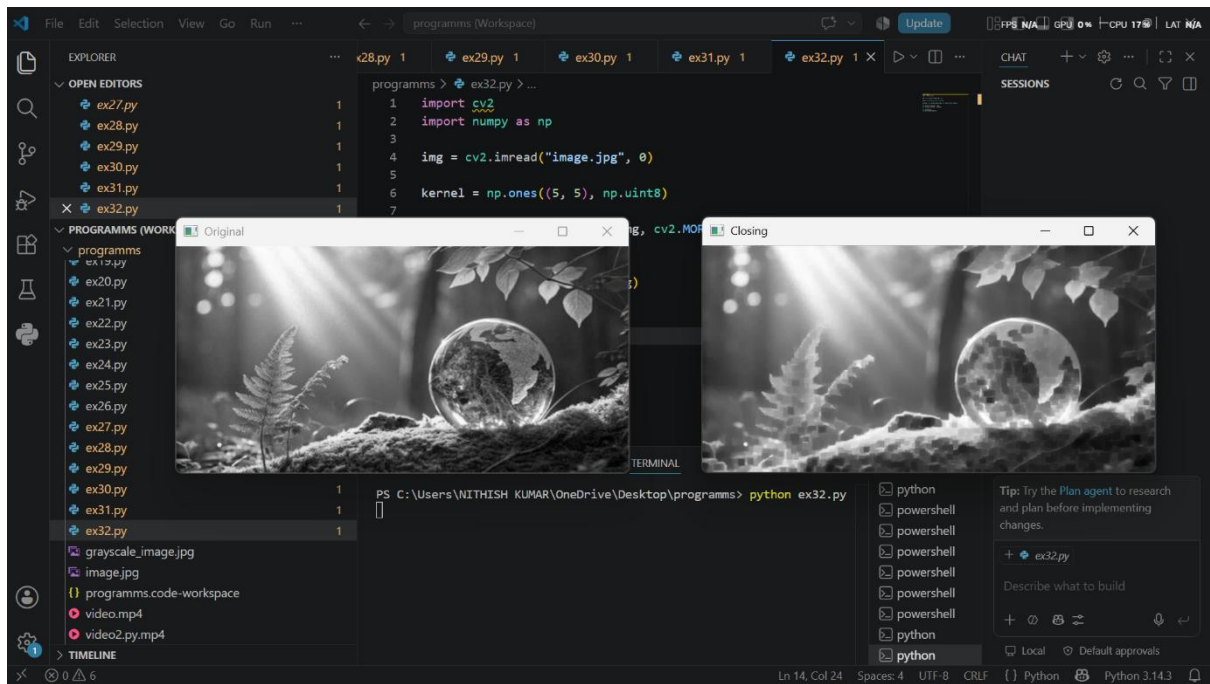
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Create a structuring element.
3. Perform dilation followed by erosion.
4. Display the output.

Result

Small holes and gaps were successfully filled.



33. Morphological Gradient

Aim

To detect object boundaries using morphological gradient.

Program

```
import cv2

import numpy as np

img = cv2.imread("image.jpg", 0)

kernel = np.ones((5, 5), np.uint8)

gradient = cv2.morphologyEx(
    img, cv2.MORPH_GRADIENT, kernel
)

cv2.imshow("Gradient", gradient)

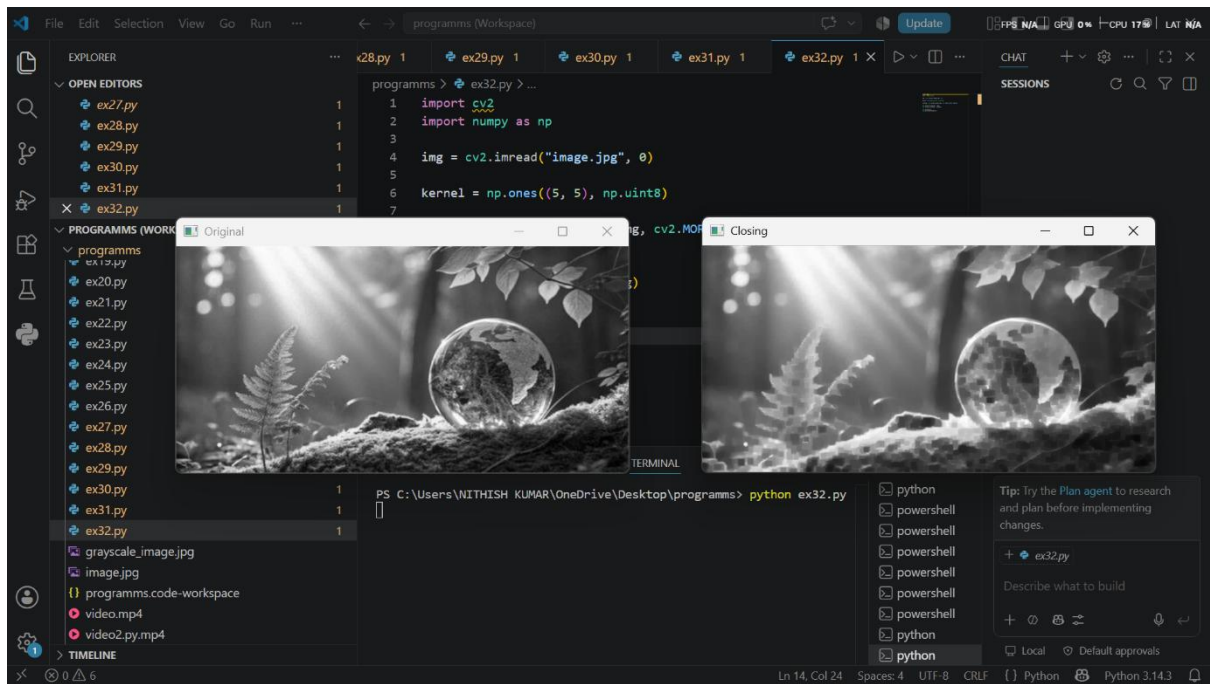
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Create the structuring element.
3. Apply morphological gradient.
4. Display the boundaries.

Result

The boundaries of objects were successfully highlighted.



34. Morphological Top Hat

Aim

To extract small bright regions using the Top Hat operation.

Program

```
import cv2

import numpy as np

img = cv2.imread("image.jpg", 0)

kernel = np.ones((5, 5), np.uint8)

tophat = cv2.morphologyEx(
    img, cv2.MORPH_TOPHAT, kernel
)

cv2.imshow("Top Hat", tophat)

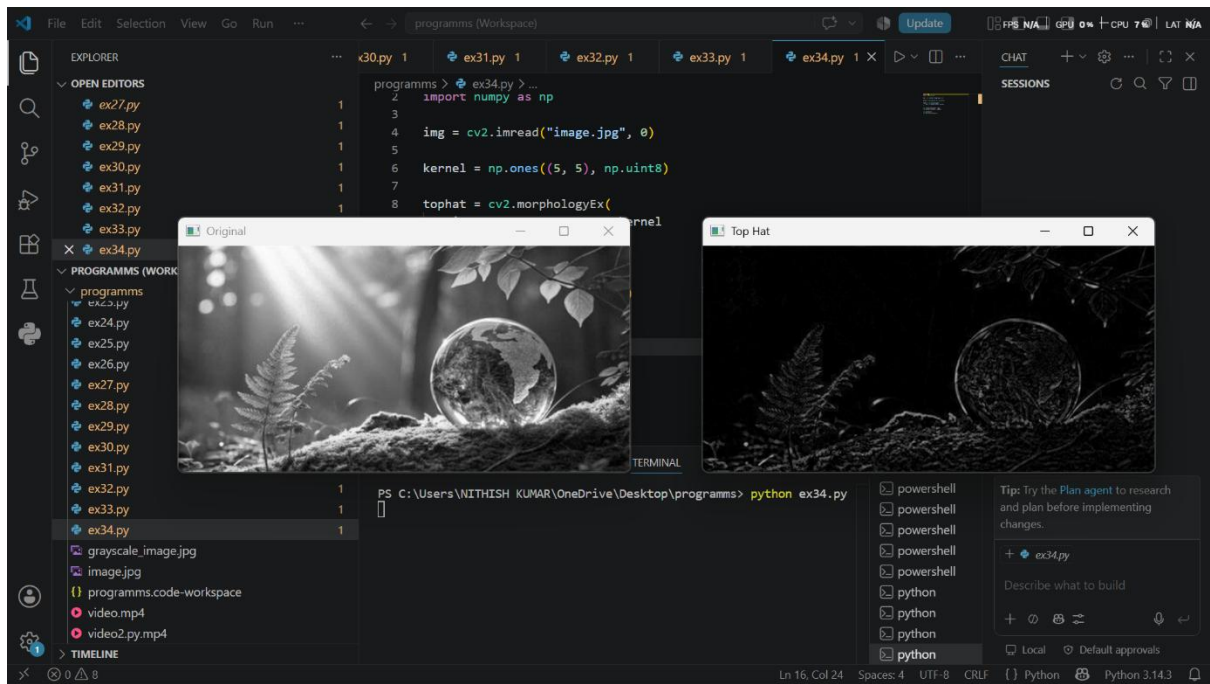
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Create a structuring element.
3. Apply Top Hat transformation.
4. Display the result.

Result

Small bright regions were successfully extracted.



35. Morphological Black Hat

Aim

To extract small dark regions using the Black Hat operation.

Program

```
import cv2

import numpy as np

img = cv2.imread("image.jpg", 0)

kernel = np.ones((5, 5), np.uint8)

blackhat = cv2.morphologyEx(
    img, cv2.MORPH_BLACKHAT, kernel
)

cv2.imshow("Black Hat", blackhat)

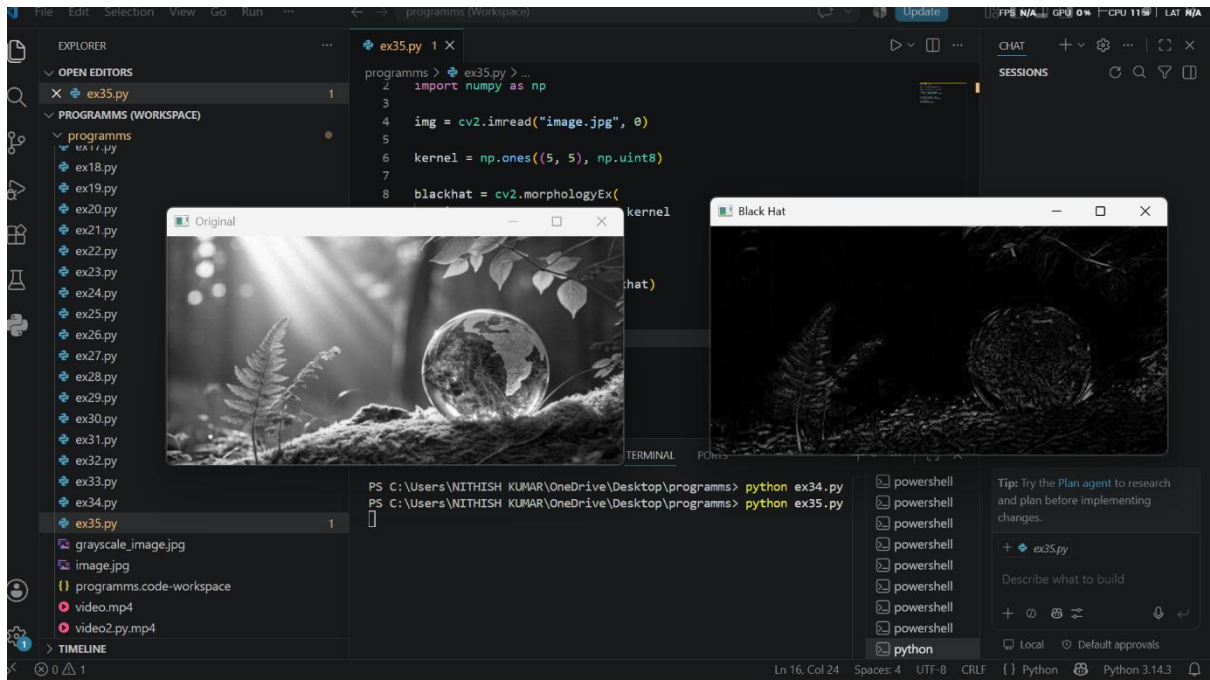
cv2.waitKey(0)
cv2.destroyAllWindows()
```

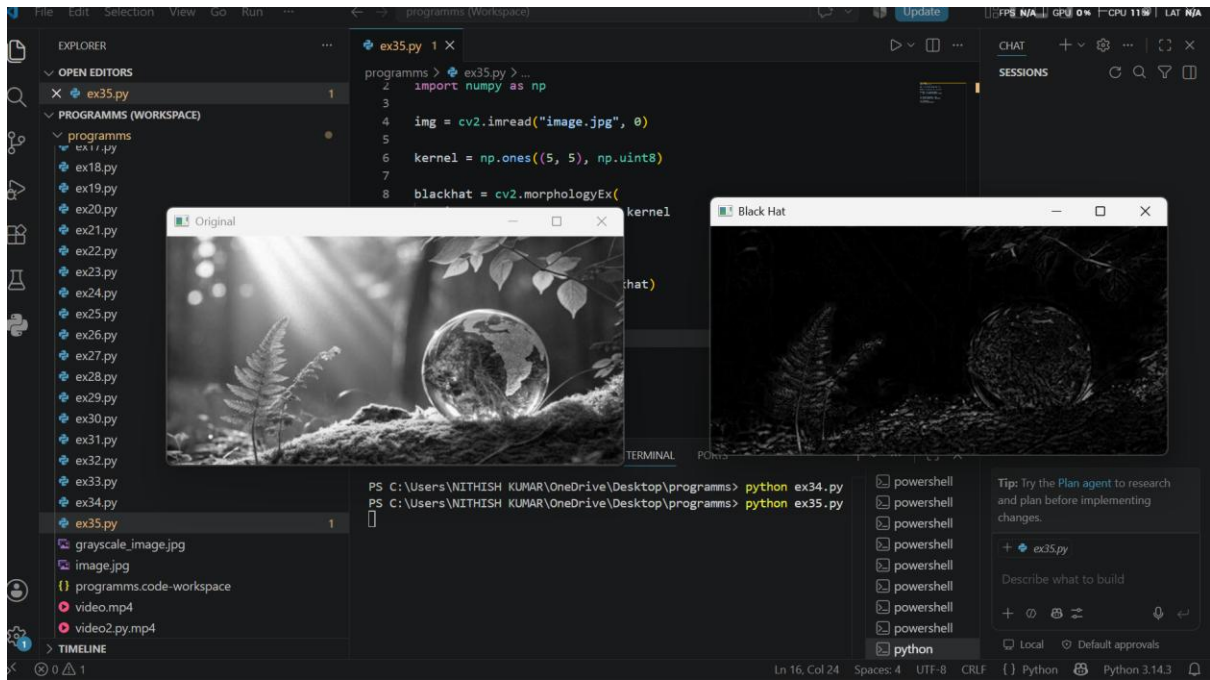
Procedure

1. Read the image.
2. Create a structuring element.
3. Apply Black Hat transformation.
4. Display the result.

Result

Small dark regions were successfully extracted.





36. Object Recognition – Watch

Aim

To recognize a watch in an image using OpenCV.

Program

A simple and practical way for this lab is **template matching**. Keep the watch template as watch.jpg.

```
import cv2
```

```
img = cv2.imread("image.jpg")
```

```
template = cv2.imread("watch.jpg")
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
temp = cv2.cvtColor(template, cv2.COLOR_BGR2GRAY)
```

```
result = cv2.matchTemplate(  
    gray, temp, cv2.TM_CCOEFF_NORMED  
)
```

```
_, max_val, _, max_loc = cv2.minMaxLoc(result)
```

```
h, w = temp.shape
```

```
if max_val > 0.6:
```

```
    x, y = max_loc
```

```
    cv2.rectangle(  
        img, (x, y),  
        (x+w, y+h),
```



```
(0, 255, 0), 2  
)
```

```
cv2.putText(  
    img, "Watch",  
    (x, y-10),  
    cv2.FONT_HERSHEY_SIMPLEX,  
    0.8, (0, 255, 0), 2  
)
```

```
cv2.imshow("Watch Recognition", img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Procedure

1. Read the input image and watch template.
2. Convert both to grayscale.
3. Perform template matching.
4. Find the best matching location.
5. Draw a rectangle around the watch.

Result

The watch was successfully recognized using template matching.

37. Play Video in Reverse Mode

Aim

To play a video in reverse order using OpenCV.

Program

```
import cv2

cap = cv2.VideoCapture("video.mp4")

frames = []

while True:
    ret, frame = cap.read()

    if not ret:
        break

    frames.append(frame)

cap.release()

for frame in reversed(frames):

    cv2.imshow("Reverse Video", frame)

    if cv2.waitKey(30) & 0xFF == ord('q'):
        break
```

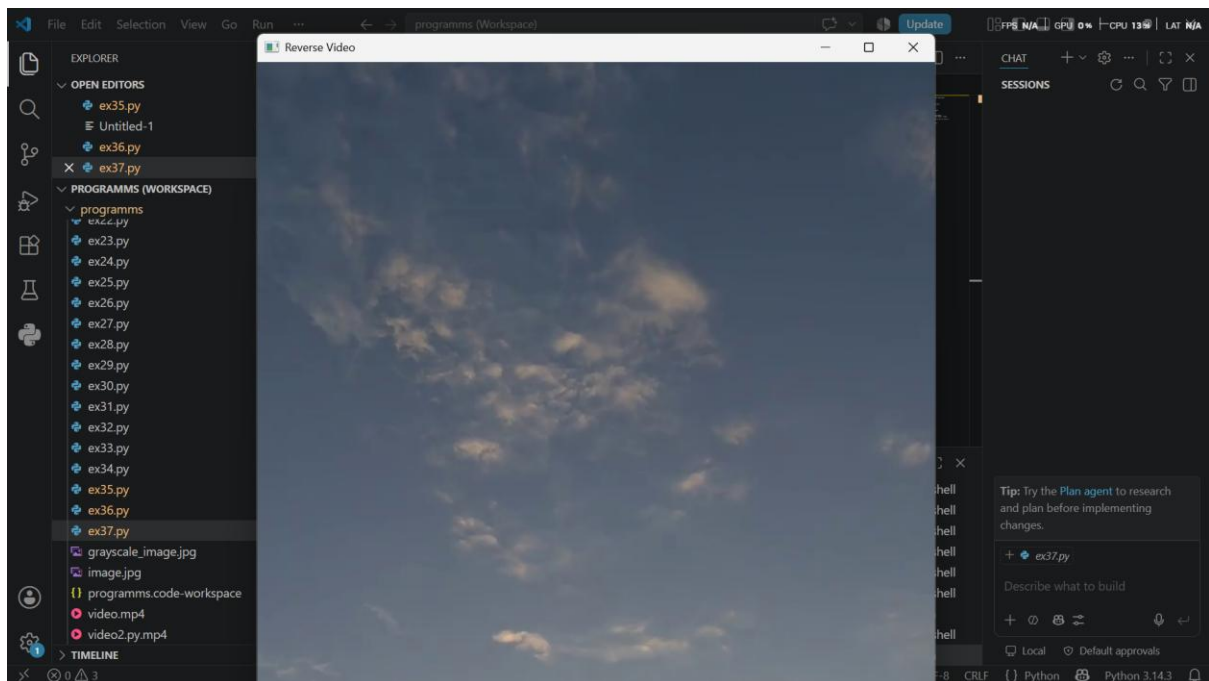
cv2.destroyAllWindows()

Procedure

1. Open the video.
2. Read and store all frames.
3. Reverse the frame sequence.
4. Display the frames.
5. Press q to stop.

Result

The video was successfully played in reverse mode.



38. Face Detection using OpenCV

Aim

To detect human faces in an image using OpenCV.

Program

```
import cv2

img = cv2.imread("image.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

face_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades +
    "haarcascade_frontalface_default.xml"
)

faces = face_cascade.detectMultiScale(
    gray, 1.1, 5
)

for (x, y, w, h) in faces:

    cv2.rectangle(
        img, (x, y),
        (x+w, y+h),
        (0, 255, 0), 2
    )

cv2.imshow("Face Detection", img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Convert it to grayscale.
3. Load the Haar Cascade classifier.
4. Detect faces using detectMultiScale().
5. Draw rectangles around the detected faces.

Result

Human faces were successfully detected and highlighted.

39. Vehicle Detection in Video

Aim

To detect moving vehicles in video frames using OpenCV.

Program

```
import cv2

cap = cv2.VideoCapture("video.mp4")

bg = cv2.createBackgroundSubtractorMOG2()

while True:

    ret, frame = cap.read()

    if not ret:
        break

    mask = bg.apply(frame)

    mask = cv2.threshold(
        mask, 200, 255,
        cv2.THRESH_BINARY
    )[1]

    contours, _ = cv2.findContours(
        mask,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
```

```
)
```

```
for contour in contours:
```

```
    if cv2.contourArea(contour) > 500:
```

```
        x, y, w, h = cv2.boundingRect(contour)
```

```
        cv2.rectangle(
```

```
            frame,
```

```
            (x, y),
```

```
            (x+w, y+h),
```

```
            (0, 255, 0), 2
```

```
        )
```

```
        cv2.putText(
```

```
            frame, "Vehicle",
```

```
            (x, y-10),
```

```
            cv2.FONT_HERSHEY_SIMPLEX,
```

```
            0.7, (0, 255, 0), 2
```

```
        )
```

```
cv2.imshow("Vehicle Detection", frame)
```

```
if cv2.waitKey(30) & 0xFF == ord('q'):
```

```
    break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

Procedure

1. Open the video.
2. Read each frame.
3. Apply background subtraction.
4. Find moving object contours.
5. Draw bounding boxes around large moving objects.
6. Press q to stop.

Result

Moving vehicles were successfully detected and marked with rectangular boxes.

40. Draw Rectangle and Extract Object

Aim

To draw a rectangular region and extract an object from an image.

Program

```
import cv2

img = cv2.imread("image.jpg")

x, y, w, h = 100, 100, 200, 200

cv2.rectangle(
    img,
    (x, y),
    (x+w, y+h),
    (0, 255, 0), 2
)

object_img = img[y:y+h, x:x+w]

cv2.imshow("Rectangle", img)
cv2.imshow("Extracted Object", object_img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Specify the rectangular coordinates.
3. Draw the rectangle using cv2.rectangle().

4. Extract the selected region using slicing.
5. Display the extracted object.

Result

The object was successfully selected using a rectangular region and extracted.

